Product Catalog API — Practice Set P1

Web Development Assessment: Backend API without a Database

Total: 100 marks (100 automated test cases, 1 mark each)

1. Problem statement

An online store needs a back-end service to manage its product catalogue. Build a REST API for a **Product Catalog** resource from scratch. You may use **Express (Node.js)** or **Spring Boot (Maven)**. There is no starter code.

Data is kept **in memory only** (an array or list inside your program). Do not use a database, a JSON file or any seed data: when your server starts, it must be **empty**. The evaluator will create every record it needs by calling your `POST` endpoint.

2. Ground rules

1. Build from scratch. Only the API is required, no frontend.
2. Your server must read its port from the `PORT` environment variable (default 8080 if it is not set).
3. Every request and response body is JSON.
4. Restart your server before each evaluation run so the data starts empty.
5. Do not change field names, paths or status codes. They are checked exactly.

3. Resource: `products`

Field	Type	Rules	Required
`id`	server-generated	Read-only. Created by your server, unique, number or string. Never taken from the request body.	—
`name`	string	Trimmed; 3 to 60 characters. Searchable and sortable.	Yes
`brand`	string	Trimmed; 2 to 40 characters. Searchable.	Yes
`category`	string (enum)	Exactly one of: `Electronics`, `Books`, `Clothing`, `Home`, `Sports`. Case-sensitive. Filterable.	Yes
`price`	number	Greater than 0 and at most 100000. Decimals allowed. Range-filterable and sortable.	Yes
`stock`	integer	Whole number from 0 to 10000. Sortable.	Yes
`inStock`	boolean	`true` or `false` (a JSON boolean, not a string). Filterable.	Yes

Unknown fields sent in a request body are ignored (they are not stored and not returned).

Sample record you would send with `POST`:

```
{"name": "Wireless Mouse", "brand": "Logitech",
  "category": "Electronics", "price": 799.5,
  "stock": 40, "inStock": true}
```

Sample record your server returns:

```
{"id": 1, "name": "Wireless Mouse",
  "brand": "Logitech", "category": "Electronics",
  "price": 799.5, "stock": 40, "inStock": true}
```

4. Endpoints

Method	Path	Purpose	Success
POST	`/api/products`	Create a record	`201` + the created record
GET	`/api/products`	List records (search, filter, sort, pagination)	`200` + list response
GET	`/api/products/{id}`	Get one record	`200` + the record
PUT	`/api/products/{id}`	Replace a record (all required fields)	`200` + the updated record
PATCH	`/api/products/{id}`	Update some fields	`200` + the updated record
DELETE	`/api/products/{id}`	Delete a record	`204` with an empty body

- `PUT` replaces the whole record and requires every required field. It never creates a record: an unknown id returns `404`.
- `PATCH` needs at least one recognised field (an empty body `{}` is `400`). Every field you send is validated by the same rules as in `POST`.
- `id` can never be changed, even if the request body contains one.

5. List query parameters (`GET /api/products`)

Parameter	Meaning	Rules
`page`	Page number	Positive integer. Default 1. Anything else is `400`. A page past the last one returns `200` with empty `data`.
`limit`	Items per page	Positive integer, **maximum 50**. Default `10`. Anything else is `400`.
`search`	Text search	Case-insensitive "contains" match on `name` **or** `brand`. Empty value means no search.
`category`	Filter by enum	Exact value from the list above. An unknown value is `400`.
`inStock`	Filter by boolean	`true` or `false`. Anything else is `400`.
`minPrice`	Lowest `price`	Inclusive. Must be a number, otherwise `400`.
`maxPrice`	Highest `price`	Inclusive. Must be a number, otherwise `400`. If `minPrice` is greater than `maxPrice`, return an empty result with `200`.
`sort`	Sort field	One of `name`, `price`, `stock`. Anything else is `400`. Text is compared ignoring letter case.
`order`	Sort direction	`asc` (default) or `desc`. Anything else is `400`.

How a list request is processed: **filters -> search -> sort -> pagination**. All filters and the search are combined with AND. Without `sort`, records come back in the order they were created.

List response:

```
{ "data": [
  {"id": 1, "name": "Wireless Mouse",
    "brand": "Logitech", "category": "Electronics",
    "price": 799.5, "stock": 40, "inStock": true}
  ],
  "page": 1, "limit": 10, "total": 1, "totalPages": 1 }
```

`totalPages` is `ceil(total / limit)`, and `0` when `total` is `0`. `total` is the number of records **after** filters and search, before pagination.

> **Short on time? Minimum list.** If you do not finish pagination, filtering, search and sort, still return `{ "data": [the first 10 records in creation order], "page": 1, "limit": 10, "total": <number of records>, "totalPages": <ceil(total / 10)> }` from `GET /api/products`. This alone earns the **List basics** marks (8 marks), and the other **60 marks** (every group except Pagination, Filter, Search, Sort and Combined queries) do not depend on these features.

6. Errors and status codes

Every error response must be JSON with a short text message under the key `error` **or** `message`, for example `{"error": "Record not found"}` or `{"message": "Record not found"}`. The **wording is free** and is not checked. The **status code is checked exactly**.

Situation	Status
Created	`201`
Read, list, update	`200`
Deleted	`204` (empty body)
Invalid body, invalid field, malformed JSON, invalid query parameter	`400`
Unknown id, or unknown route	`404`
Duplicate unique value	`409`

Validation: strings must be JSON strings (trimmed; blank after trimming is invalid), numbers must be JSON numbers, booleans must be JSON booleans (`"true"` as text is invalid), integers must be whole numbers, and enums must match exactly.

- This resource has no unique field. Two records with identical values are allowed. Your server must never crash or return an HTML error page, even for malformed JSON.

Rules that are easy to miss

1. An id that does not exist returns `404`, whatever it looks like (`/api/products/987654`, `/api/products/abc`). Never `400` and never `500`.
2. Text is trimmed before it is checked and before it is stored: `"  Wireless Mouse  "` is saved as `"Wireless Mouse"`.
3. A number sent as text (`"799.5"`) is invalid. So is `inStock` sent as `"true"`, and `stock` sent as `1.5`.
4. `price` of `0` or a negative number is invalid.
5. `POST` ignores any `id` you send. `PUT` and `PATCH` never change the id.
6. `PUT` on an unknown id is `404`. It does **not** create a record.
7. `PATCH` with `{}` is `400`. `PATCH` leaves every field you did not send unchanged.
8. `DELETE` returns `204` with **no body**. Deleting the same id again returns `404`.
9. A page past the last page is **not** an error: return `200` with `"data": []`.
10. `minPrice` greater than `maxPrice` is **not** an error: return `200` with `"data": []`, `total` `0`, `totalPages` `0`.
11. Without `sort`, records come back in the order they were created (oldest first).
12. Query values you do not support (`sort=banana`, `order=sideways`, `inStock=maybe`, a category that is not in the list) are `400`.

7. Examples

All examples use the base path `/api/products`. Values are illustrative.

Create, invalid (`price` is 0): `POST /api/products` returns `400`

```
{ "error": "price must be greater than 0" }
```

Get one, unknown id: `GET /api/products/987654` returns `404`

```
{ "error": "Record not found" }
```

Filter, sort and paginate: `GET /api/products?category=Electronics&inStock=true&sort=price&order=desc&page=1&limit=2` returns the 2 records with the highest `price` among those whose `category` is `Electronics` and `inStock` is `true`. `total` is the number of matching records, not 2.

```
{ "data": [
    {"id": 3, "name": "Wireless Mouse",
      "brand": "Logitech",
      "category": "Electronics", "price": 799.5,
      "stock": 40, "inStock": true}
  ],
  "page": 1, "limit": 2, "total": 5, "totalPages": 3 }
```

Only one record is shown to keep the example short: `data` holds up to `limit` records.

Search: `GET /api/products?search=wireless` returns every record whose `name` or `brand` contains that text in any letter case.

Range: `GET /api/products?minPrice=400&maxPrice=1599` returns records with `price` from 400 to 1599, both ends included.

8. How your work is evaluated

- Your server is started **empty**. The evaluator then creates its own records by calling `POST /api/products` (about 45 records with random values, **not** the sample above), and runs **100 automated checks** on every endpoint in a fixed order.
- Each check is worth **1 mark**. There are no part marks inside a check.
- Checks look at the HTTP status code, the JSON shape and the field values. They do not look at your code, folder structure, language or framework (Express and Spring Boot are marked the same way).
- Error wording is not checked (see section 6). Status codes and success bodies are checked exactly.
- If `POST` does not work, most other checks cannot run and their marks are lost. **Build and test `POST` first.**
- If your server crashes while the checks run, every check after the crash fails. Handle bad input without crashing.
- Each request must answer within 5 seconds. A full run is limited to 3 minutes.
- Because data is random, you cannot pass by hard-coding answers.

Suggested build order: `POST` and `GET /{id}`, then `GET` list, `PUT`, `PATCH`, `DELETE`, then pagination, filters, search and sort, then a pass through the "rules that are easy to miss".

9. Test your API yourself (practice sets)

No evaluator is given for practice sets. Test your API with Postman, Thunder Client or curl. Start your server empty, and work through this list. On assessment day the same kinds of checks are run by an evaluator, with different data.

#	Request	Expect
1	`POST /api/products` with the sample record	`201`, body has an `id`
2	`POST` with `name` removed	`400`
3	`POST` with `price` set to `0`	`400`
4	`POST` with `price` sent as text (`"100"`)	`400`
5	`POST` with `inStock` set to `"yes"`	`400`
6	`POST` with `category` set to `"Banana"`	`400`
7	`POST` with a broken body (`{`)	`400`, JSON error, server still running
8	`GET /api/products`	`200`, `data`, page `1`, `limit` `10`, `total`, `totalPages`

#	Request	Expect
9	`GET /api/products/{id}` using an id you created	`200`, the same values
10	`GET /api/products/987654` and `GET /api/products/abc`	`404` for both
11	PUT a full record on a real id	`200`, new values, same id
12	PUT with a field missing; PUT on an unknown id	`400`; `404` (nothing created)
13	PATCH only `inStock`	`200`, only that field changed
14	PATCH with `{}`	`400`
15	DELETE a real id, then GET it, then DELETE it again	`204` with empty body, `404`, `404`
16	Create 12 records, then `?page=2&limit=5`	items 6 to 10, `totalPages` 3
17	`?limit=51`, `?limit=0`, `?page=0`, `?page=abc`	`400` each
18	`?category=Banana`, `?inStock=maybe`	`400` each
19	`?search=` the same word in UPPER and lower case	same `total`
20	`?sort=banana`, `?order=sideways`	`400` each
21	`?minPrice=100&maxPrice=10`	`200`, empty `data`, `total` 0
22	`GET /api/nothing`	`404`, JSON body

10. Marking scheme

Group	What is checked	Marks
Create (POST)	Valid create returns 201 with an id and the stored values (text trimmed); rejects missing fields, wrong types, out-of-range numbers, short text, bad enum values, empty body and malformed JSON with 400; duplicate rule for the unique field.	14
Read one (GET /:id)	Returns the right record for each id; 404 with a JSON error body for unknown ids, including ids that do not look like ids.	8
List basics (GET)	200 and the list shape (data, page, limit, total, totalPages); defaults page 1 and limit 10; correct total; every item well formed; creation order by default.	8
Update (PUT)	Full replace returns 200 with new values and the same id; 400 for missing or invalid fields; 404 for unknown id (nothing is created); other records untouched; id in the body ignored; duplicate rule.	10
Update (PATCH)	Partial update changes only the fields sent; 400 for an empty body, invalid values and wrong types; 404 for unknown id; unknown fields ignored; id cannot change; duplicate rule.	10
Delete (DELETE)	204 with an empty body; the record is gone afterwards; second delete and unknown id return 404; total drops by exactly one; other records unaffected.	8
Pagination	limit and page give the right slices without overlap; last page holds the remainder; totalPages; every record appears exactly once across pages; page past the end is empty; invalid page and limit values return 400; limit above 50 returns 400.	12
Filter	Enum filter, true/false filter and the min/max range (inclusive) return exactly the matching records; filters combine with AND and with pagination; invalid filter values return 400.	10
Search	Case-insensitive partial match on the two text fields; no match gives an empty list with total 0; works with pagination; empty search means no search.	8
Sort	Ascending and descending order on the allowed fields (numbers sorted as numbers, text ignoring case); invalid sort field or order returns 400.	6
Combined queries	Search, filter, sort and pagination together give the exact expected page; min greater than max gives an empty result with 200.	4
HTTP basics	JSON content type on every response and a JSON 404 for unknown routes.	2
Total		**100**